

Experiment 04:

Laboratory Exercise

Inputs to the Function:

- ❖ **Graph** : represents the nodes and their neighbouring nodes with edge costs.
- ❖ **Heuristic** : contains the heuristic value $h(n)$ for each node.
- ❖ **start_node** : represents the source node from where the search begins.
- ❖ **goal_node** : represents the destination node that has to be reached.

Code:

```
def a_star_search(graph, heuristic, start_node, goal_node):  
  
    # Step 1: Start from the source node  
  
    # Step 2: Set  $g(\text{start}) = 0$   
    g_cost = {  
        start_node: 0  
    }  
  
    # Step 3: Calculate  $f(\text{start}) = g(\text{start}) + h(\text{start})$   
    f_cost = {  
        start_node: g_cost[start_node] + heuristic[start_node]  
    }  
  
    # Step 4: Store the start node in the Open List  
    open_list = [start_node]  
  
    # Closed List  
    closed_list = []  
  
    # Store the path  
    came_from = {  
        start_node: None  
    }  
  
    while open_list:  
  
        # Step 5: Select the node having the smallest  $f(n)$   
        current_node = min(  
            open_list,  
            key=lambda node: f_cost[node]  
        )
```

```
# Check whether goal is reached
if current_node == goal_node:

    path = []

    while current_node is not None:
        path.append(current_node)
        current_node = came_from[current_node]

    path.reverse()

    return path, g_cost[goal_node]

# Step 6: Expand neighboring nodes
for neighbor_node, edge_cost in graph[current_node]:

    if neighbor_node in closed_list:
        continue

    # Calculate new g(n)
    new_g_cost = (
        g_cost[current_node] + edge_cost
    )

    # Step 7: Update cost if a better path is found
    if (
        neighbor_node not in g_cost
        or new_g_cost < g_cost[neighbor_node]
    ):

        g_cost[neighbor_node] = new_g_cost

    # Step 3: Calculate f(n) = g(n) + h(n)
    f_cost[neighbor_node] = (
        g_cost[neighbor_node]
        + heuristic[neighbor_node]
    )

    came_from[neighbor_node] = current_node

    # Step 4: Add node to Open List
    if neighbor_node not in open_list:
        open_list.append(neighbor_node)

# Step 8: Move explored node to Closed List
open_list.remove(current_node)
closed_list.append(current_node)

# No path found
return None, float("inf")
```

Defining Graph & Assigning Heuristic Values:

```
start_node = "A"
goal_node = "G"

shortest_path, total_cost = a_star_search(
    graph,
    heuristic,
    start_node,
    goal_node
)

print("Shortest path:", " -> ".join(shortest_path))
print("Total cost:", total_cost)
```

Set the Start Node & Goal Node:

```
graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("D", 3), ("C", 2)],
    "C": [("E", 5)],
    "D": [("G", 4), ("F", 2)],
    "E": [("G", 3)],
    "F": [("G", 1)],
    "G": []
}

heuristic = {
    "A": 5,
    "B": 6,
    "C": 4,
    "D": 3,
    "E": 3,
    "F": 1,
    "G": 0
}
```

Output:

```
... Shortest path: A -> B -> D -> F -> G
    Total cost: 7
```

Conclusion:

The A* algorithm was successfully implemented to find the shortest path between a given source node and goal node. It uses $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost and $h(n)$ is the heuristic estimate. By maintaining Open and Closed Lists and selecting the node with the smallest $f(n)$, the algorithm efficiently finds the optimal path.